Group symbol: **2**

Team: **1**

Project title: **ListItUp**

Team members (*filled by PM, Team Leader*):

No	Name	Surname	Student ID	Role
1	Carmelo Aidan	Romero Pérez	293936	*PM, Team Leader*
2	Álvaro	Puebla Ruisánchez	293867	*Team member*
3	Carmen	Gutiérrez Sánchez	293906	*Team member*
4	David	Sosa Domínguez	293885	*Team member*
5	Diego	García Agrelo	293939	*Team member*
6	Miryam	Merchán León	293907	*Team member*

1. Elaboration of application concept (S1)

1.1. Project (business) goals

Present as a tree of the project's objectives, i.e., decomposition of general project objective into specific ones. A goal should be:

- relevant to the project,
- important from the perspective of the end-user,
- achievable,
- measurable,
- realistic (achievable in terms of constraints: cost and time-oriented, technological, organizational, human resources, legal).

General objective: Build a web platform that allows users to easily create, organize, and share curated lists of their favorite things (like products, places, tools, or movies). The focus is on building a fun, community-driven space where users can discover cool recommendations and share their own top picks.

The **main goals** are:

1. Easy List Creation

- Provide a simple interface so anyone can create and publish a list in just a few minutes.
- Allow users to add items with basic details like a title, image, short description, and an external link.
- Let users organize their lists into categories (e.g., Tech, Food, Travel) and save drafts.
- Measure: At least 80% of our test users (classmates and friends) can successfully create and publish a list without asking for help.

2. Discovery Engine

- Create a homepage feed where users can see trending lists or browse by category.
- Implement a basic search function so users can find specific lists or tags quickly.
- Enable filtering and sorting by category, popularity, and recency.
  Measure: A new user should be able to find an interesting list within 10-15 seconds of opening the app.

3. Social and Community Features

- Allow users to follow their friends or favorite creators to see their new lists.
- Enable saving lists to personal collections and receiving update notifications when saved lists are modified.
- Support reactions (likes/upvotes) and threaded comments on lists to foster engagement.
- Create basic user profiles showing their published lists, follower count, and saved collections.
- Measure: Achieve at least one interaction (like, save, or comment) from 15% of the users testing our app during the evaluation phase.

4. Platform Performance, Reliability, and Accessibility

- Support mobile, tablet, and desktop browsers without requiring a native application installation.
- Keep page load times under 2-3 seconds for a smooth user experience.
- Ensure system availability of at least 99.5% during the MVP pilot period.
- Implement basic content moderation (like a "report" button) so we can easily remove inappropriate test content.

5. Sustainable Technical Foundation

- Implement a modular, API-first architecture enabling independent scaling of front-end and back-end components.
- Maintain basic automated tests for the most critical parts of our backend to avoid breaking the app during development.
- Document our API and database models clearly so any team member can understand them.

1.2. Identification of project's internal and external Stakeholders

Identify the role of each **internal** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
I1	Carmelo A. Romero	Cross-functional team member	Leads the team, coordinates stakeholders, owns schedule and scope, primary contact for sponsor/IT; also contributes across project tasks as needed.
I2	Álvaro Puebla	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I3	David Sosa	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I4	Carmen Gutiérrez Sánchez	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I5	Diego García	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I6	Miryam Merchán León	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.

Identify the role of each **external** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
E1	Casual Users	Standard Users	Everyday users who use the app to discover cool recommendations and share their own interests. They can browse, create, edit, and share their own lists. They interact with the community by leaving likes, comments, and saving other people's lists to their personal collections.
E2	Verified Creators	Content Creators/Influencers	High-profile users, influencers, or experts who have a "verified" status on the platform. Alongside standard features, they have access to exclusive tools: a "Creator Dashboard" to view profile and list analytics (like views, saves, and link clicks), a verified badge on their profile, and the ability to pin top lists to their page.
E3	App Administrators	Platform Moderators	A special user role (managed by our team during this phase) with permissions to manage the platform. They can delete inappropriate test lists, remove buggy comments, and keep the database clean during the testing period.
E4	Third-Party API Providers	Technology Partners	Providers of external APIs integrated into the platform: social login (Google, GitHub), image hosting, and link-metadata enrichment. They define technical constraints and service-level agreements.

1.3. Domain description

- Description of phenomena in the application domain that can be observed.
- Definitions of concepts that abstract these phenomena.
- The concepts include:
    - entities - things that one can “distinguish” from the fraction of reality,
    - relationships - either properties (unary relationships of entities) that yield some information about them or relationships between two or more entities (binary or *n*-ary relations),
    - events - things that can happen (occur).
- Use UML class diagram to depict entities (class with attributes), relationships (associations).
- This diagram **must** be supplemented with the list of entities, relationships and events along with their description in natural language.

Domain Phenomena:

- **User Registration & Roles**: Users sign up to the platform. They can be standard users, "verified" creators (influencers/VIPs), or admins who manage the platform.
- **Creating and Managing Lists**: Users build collections of their favorite things (e.g., top 10 movies, best study tools). A list can be public or private, and the creator can edit or update it over time.
- **Adding Items:** Inside each list, users add specific items. Each item usually has a title, a short description, an image, and an external link to the product or place.
- **Social Interactions and Discovery:** Users browse the app's feed or search by categories to discover new lists. If they find an interesting list, they can leave a "Like", write a comment, or save the list to their personal profile to check it out later.
- **Follower System and Creator Stats:** Users can follow their favorite creators to see their new lists on their home feed. Verified creators get access to extra data, like how many views or saves their lists are getting.
- **Follow graph:** Users follow curators, creating a directed social graph. Followers receive a personalized feed of newly published or updated lists from those they follow.
- **Alerts and Notifications:** The app sends a simple notification when someone likes a user's list, leaves a comment, or when a followed creator posts something new. Also, if there is an update to a saved list, trigger notifications to relevant users via in-app and/or email channels.
- **Content moderation:** Users can report lists, items, or comments that violate platform guidelines. Reported content enters a moderation review queue managed by administrators.

Definition of Concepts:

1. **User** - A registered account on the platform.
    - *Key attributes*: (userId, username, email, password, biography, profilePicture, role, createdAt, |standard, verified, admin|)

2. **List** - A list of items made by an user.
    - *Key attributes*: (listId, title, description, coverPhoto, createdAt, visibility |public, private|)

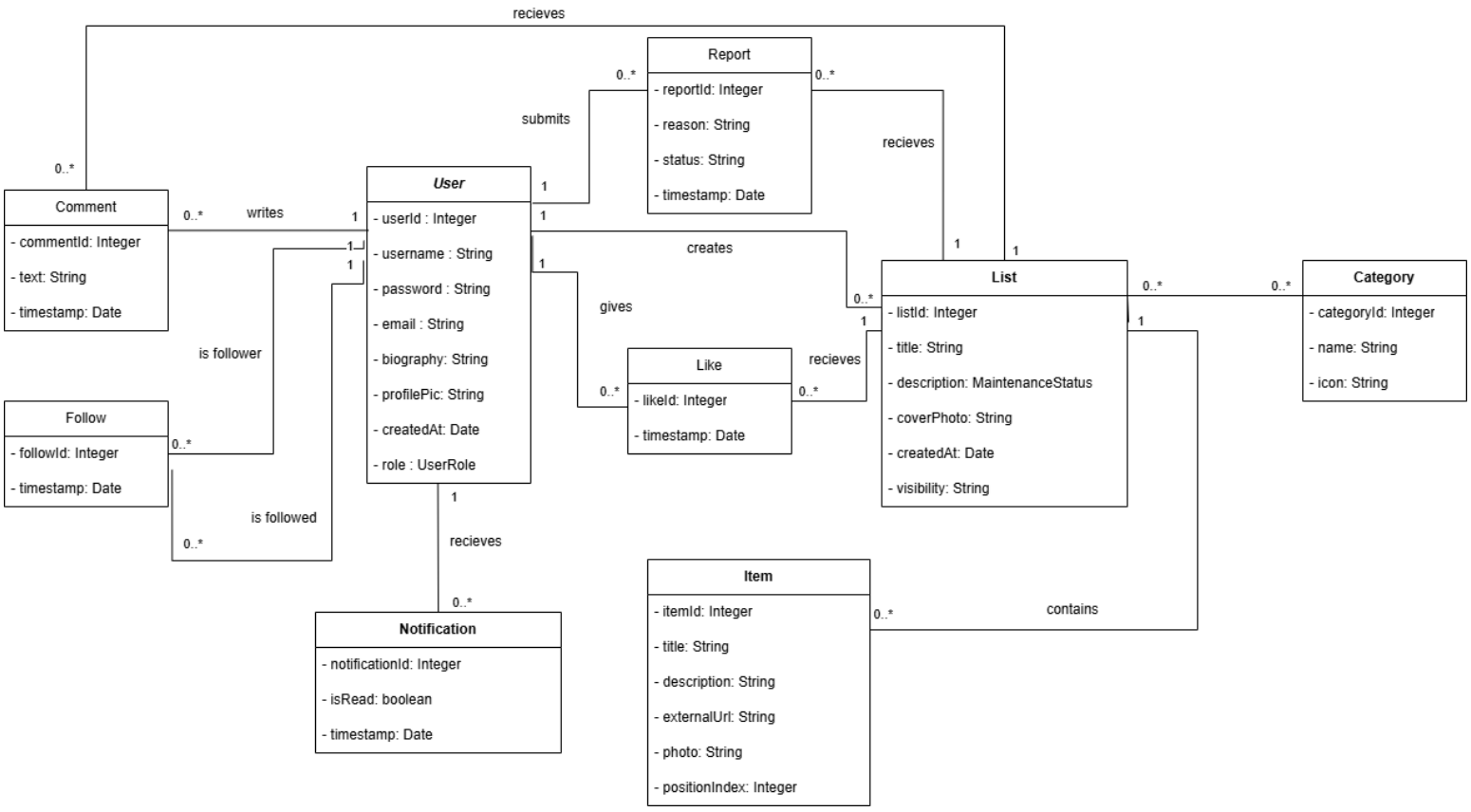
3. **Item** - A single entry within a List.
    - *Key attributes*: (itemId, title, description, externalUrl, photo, positionIndex,)

4. **Category** - To group lists and find them on the feed.
    - *Key attributes*: (categoryId, name, icon)

5. **Comment -** A textual contribution by a user on a CuratedList.
    - *Key attributes*: (commentId, text, time_stamp)

6. **Notification** - A system message generated by a domain event.
    - *Key attributes*: (notificationId, message, is_read, timestamp)

7. **Report** - a user-submitted content moderation request.
    - *Key attributes*: (report_id, reason, status |open, reviewed, resolved|, timestamp).



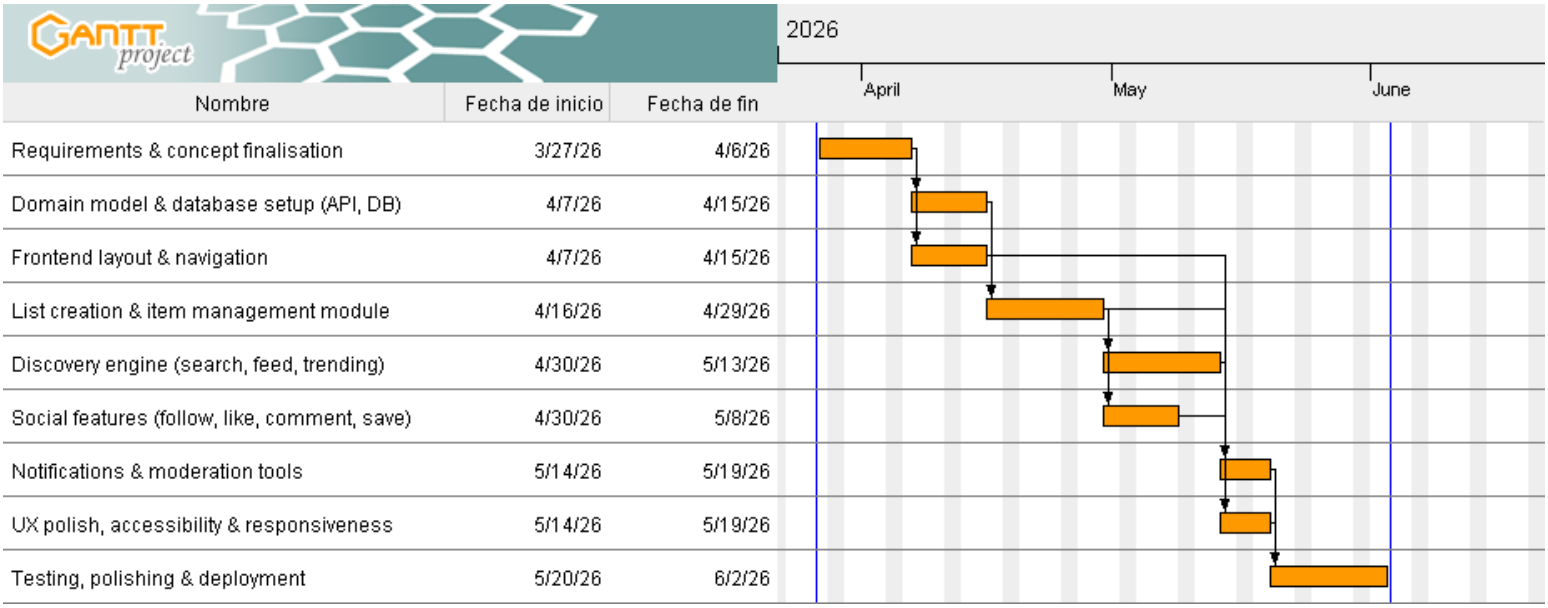
Class Diagram

1.4. Project schedule (Gantt chart)

- tasks with dependencies,
- start time and duration,
- resource allocation: staff, costs, equipment.

Overview: 7-week MVP. Tasks include dependencies, start week (W1 = project start), duration in weeks, primary assigned role, and estimated staff-hours.

ID	Task	Duration	Depends on	Notes
T1	Requirements & concept finalisation	1 wk	-	Define list types and workflows
T2	Domain model & database setup (API, DB)	1 wk	T1	Entities, relations, policies
T3	Frontend layout & navigation	1 wk	T1	React base, pages, component library
T4	List creation & item management module	1.5 wk	T2	Curator editor, draft/publish flow
T5	Discovery engine (search, feed, trending)	1.5 wk	T4	Full-text search, trending, personalised feed
T6	Social features (follow, like, comment, save)	1 wk	T4	Follow graph, interactions, saved collections
T7	Notifications & moderation tools	0.5 wk	T5, T6	Follow graph, interactions, saved collections
T8	UX polish, accessibility & responsiveness	0.5 wk	T3-T6	WCAG 2.1 AA pass, mobile review
T9	Testing, polishing & deployment	1.5 wk	T7, T8	QA + Docker deploy



Team effort (approx.): 384 hours
Estimated cost (approx.): €7,000 - €10,000
Resources: shared developer laptops; cloud database and hosting (free/low-cost tiers for MVP); third-party free-tier services for OAuth, image hosting, and email delivery.

1.5. Identification of existing or alternative solutions

Each competitor should be characterized with

- name of the project/application,
- main features,
- advantages,
- disadvantages.

Are.na

- **Main features:** Collaborative, non-algorithmic content curation tool. Users create "channels" containing any type of block (text, image, link, file) and can connect blocks across channels to form visual knowledge collections.
- **Advantages:** Highly flexible, open-ended structure suited to creative professionals; strong community of designers and researchers; no recommendation algorithm - editorial control belongs entirely to the user; public API available.
- **Disadvantages:** Deliberately niche and minimalist, limiting mass adoption; steep learning curve; limited discovery features; free tier has a block limit; UX is unfamiliar to general audiences.

Raindrop.io

- **Main features:** Cross-platform bookmark manager with collections, tags, highlights, and full-text search. Supports visual collection views and public sharing of bookmark collections.
- **Advantages:** Polished, feature-rich bookmark management; excellent browser extension; clean UI; good import/export support; available across all major platforms (web, mobile, desktop).
- **Disadvantages:** Primarily a personal productivity tool rather than a social discovery platform; weak community and follow features; lacks item metadata enrichment such as curator ratings; minimal trending discovery functionality.

Product Hunt

- **Main features:** Community-driven platform for discovering new technology products. Daily launches are upvoted by users; products accumulate reviews and discussions.
- **Advantages:** High-quality community of makers and early adopters; authoritative for product discovery; strong SEO; robust notification and discussion systems; strong brand recognition in the tech space.
- **Disadvantages:** Scope limited to technology products; collection and curation features are secondary to the launch feed; strong platform lock-in for makers; not suitable for general-purpose list creation; perceived as promotional by general users.

Letterboxd

- **Main features:** Social film diary and review platform. Users log films watched, write reviews, and create thematic list collections (e.g., "Best horror films of the 1970s"). Features a strong social follow graph and discovery via lists and reviews.

- **Advantages:** Excellent domain-specific UX for film enthusiasts; high-quality curator community; elegant list-building tools; strong personal diary/log integration; large and growing active user base.
- **Disadvantages:** Entirely domain-specific (films only); not generalisable to other content types; no public API for programmatic access; limited features for professional curators outside the film domain.

Listly

- **Main features:** Web-based list creation and curation tool primarily targeting content marketers and bloggers. Users create embeddable lists of links and resources, collaboratively curated, with social sharing options.
- **Advantages:** Embeddable widget model for external content sites; collaborative editing; good SEO for list pages; easy link import.
- **Disadvantages:** Outdated and cluttered UI; limited discovery and social features; primarily focused on B2B and marketing use cases; weak mobile experience; minimal active general community.

Notion (Public Databases)

- **Main features:** All-in-one productivity workspace supporting public-facing databases, linked resources, gallery views, and filtered list publications. Often used informally as a curated-list publication tool.
- **Advantages:** Extremely flexible; excellent formatting and media embedding; no-code database views; large existing user base; free tier available.
- **Disadvantages:** Not purpose-built for social discovery; no native follow or notification model for content subscriptions; slow public page load; no recommendation engine; no community or engagement features.

1.6. Project context

- application context
- technological context
- organisational context
- legal context

Indicate aspects that are important for the product. It should lead to the identification of constraints of the project and the chances for its development.

Application Context

- General-purpose curated list discovery platform for any user who seeks trustworthy, well-organised collections of information rather than raw search results.
- Primary user needs: fast content creation, intuitive discovery, social follow graph, and timely notifications when followed curators publish new lists.
- Key use cases: a productivity enthusiast publishing a "Top 20 AI writing tools" list; a travel blogger maintaining a "Best independent cafes in Warsaw" list; a student discovering reading lists compiled by domain professionals.
- Target delivery: an MVP accessible via web browser on desktop and mobile, without requiring a native application installation.

Technological Context

- Required integrations: OAuth 2.0 social login (Google, GitHub); Open Graph metadata API for link enrichment; cloud object storage for images; transactional email service for notifications.
- Platform constraints: web-first responsive SPA; single-host Docker deployment for the MVP; HTTPS everywhere.
- Non-functional targets: page load under 2 s; full-text search response under 300 ms; system **availability ≥ 99.5%** during the pilot; WCAG 2.1 AA compliance for all core pages.
- Security baseline: OWASP Top 10 mitigations; input validation; rate limiting on public API endpoints; encrypted sensitive fields at rest; basic observability with error tracking and metrics/alerts.

Organisational Context

- Key stakeholders: casual users, curators, data protection authority, third-party API providers.
- Team of six cross-functional members operating under an agile iterative methodology with a two-week sprint cadence.
- Tooling: GitHub for version control and project tracking; Figma for design; Slack for asynchronous communication; weekly synchronous stand-ups.
- Deployment and operations: must include a simple rollback plan and documented deployment runbooks; code reviews mandatory before merging to the main branch.

SSD 2026 - Project Report S1

Legal & Compliance Context

- GDPR / RODO: a Data Protection Impact Assessment (DPIA) must be conducted before launch; all processing activities must have a clearly identified legal basis; users must be able to exercise access, rectification, erasure, and portability rights.
- Privacy Policy and Terms of Service: plain-language legal documents covering data collection, third-party integrations, user content ownership, and acceptable use must be published at launch.
- Content ownership: users retain copyright over content they create; the platform acquires a non-exclusive licence to display and distribute that content; the platform is not liable for the accuracy or availability of external links.
- Data retention: user account data retained for the active account lifetime and up to 2 years post-deletion for audit purposes; interaction logs retained for 12 months; transactional email logs for 6 months.
- Cookie consent: any cookies beyond strictly necessary session cookies require informed user consent under the ePrivacy Directive.
- Accessibility: WCAG 2.1 AA compliance required for all public-facing pages.

Security & Resilience

- Authentication & identity: OAuth 2.0 (Google, GitHub) and email/password login; enforce RBAC with role-based route protection; JWT-based stateless sessions.
- Transport & storage: TLS everywhere; encrypt sensitive fields at rest; hash passwords with bcrypt.
- Hardening & availability: pre-launch vulnerability scan; dependency update policy; health checks and maintenance mode support.
- Monitoring, backups & runbooks: error tracking and metrics/alerts; centralised structured logs; daily incremental and weekly full backups; runbooks for deploy, rollback, and third-party outage scenarios.

Constraints (must manage)

- Time: MVP pilot window 7 weeks.
- Budget: target < €10,000; prioritise open-source and containerised components.
- Devices: support modern desktop and mobile browsers; no mandatory native app.
- Recommendation sophistication: limited to engagement-based ranking in the MVP (no machine-learning model).

Opportunities (advantages to exploit)

- Growing demand for high-quality, human-curated information as an alternative to recommendation-algorithm fatigue.
- Low competitive density in the general-purpose, web-first curated list discovery space.
- Creator economy trend: curators seeking platforms to build audiences, supporting future premium curator features and monetisation.
- Rapid, low-cost MVP using containerised open-source building blocks.

Measurable Success Criteria (pilot)

- Functional: > 80% of pilot users can create and publish a list without staff assistance.
- Reliability: end-to-end list creation/publish success rate ≥ 98% in QA tests.
- Discovery: average time-to-first-relevant-list < 10 seconds for a new visitor.
- Engagement: interaction rate (like, save, or comment) of at least 15% of active monthly users.

1.7. Technologies used in the project

Justify the choice of technology. Focus on each element and provide the required information

- technology name,
- description of the technology,
- justification,
- key responsibilities in the project, e.g., front-end development,
- link to the technology description.

Java (Backend Language)

- **Description:** General-purpose, statically typed, object-oriented programming language and runtime platform, widely used for building enterprise-grade server-side applications.
- **Justification:** Java offers strong static typing, mature tooling, and a vast ecosystem of libraries for building reliable backend services. Its strict type system reduces runtime errors, and the JVM provides consistent cross-platform execution. All team members had existing experience with Java, eliminating onboarding overhead within the limited project window.
- **Key responsibilities:** Implementation of all server-side business logic - resource management, reservation and loan lifecycle, user role handling, and API endpoints.
- **Reference:** java.com - docs.oracle.com/en/java

Spring Boot (Backend Framework)

- **Description:** Opinionated Java framework built on top of the Spring ecosystem that enables rapid development of production-ready web applications and REST APIs with minimal configuration.
- **Justification:** Spring Boot drastically reduces boilerplate configuration compared to plain Spring; it provides built-in support for REST controllers, dependency injection, JPA/Hibernate ORM, security, and transaction management - all essential for a resource management system. Its embedded server (Tomcat) simplifies deployment inside Docker containers.
- **Key responsibilities:** REST API layer, business logic services, authentication and authorisation (Spring Security), database access via Spring Data JPA, and application configuration management.
- **Reference:** spring.io/projects/spring-boot

HTML + CSS + JavaScript (Frontend)

- **Description:** Standard web technologies used to build the user interface: HTML5 for document structure, CSS3 for styling and responsive layout, and vanilla JavaScript for client-side interactivity and API communication.
- **Justification:** Given the scope of the MVP and the team's existing skills, a lightweight frontend built with plain web technologies offers the fastest path to a working interface without the overhead of a JavaScript framework. The backend serves Thymeleaf-rendered pages or static HTML/CSS/JS assets directly, keeping the stack simple and the deployment straightforward.
- **Key responsibilities:** All user-facing pages - resource browsing, reservation forms, loan management views, check-in/check-out interfaces, and admin panels.
- **Reference:** developer.mozilla.org/en-US/docs/Web

MySQL (Relational Database)

- **Description:** Open-source relational database management system that stores data in structured tables with full support for SQL, ACID transactions, and foreign key constraints.
- **Justification:** MySQL is a proven, lightweight relational database well-suited to the structured, transactional nature of resource management data (reservations, loans, users, policies). It pairs natively with Spring Data JPA and Hibernate, and its configuration is straightforward to manage inside a Docker container. The mysql/conf directory in the repository provides a custom configuration for the containerised instance.
- **Key responsibilities:** Persistent storage of all domain data: users, resources, reservations, loans, check events, holds, notifications, and policies.
- **Reference:** mysql.com - dev.mysql.com/doc

Nginx (Reverse Proxy & Web Server)

- **Description:** High-performance, open-source HTTP server and reverse proxy capable of serving static files, forwarding requests to application servers, and terminating TLS connections.
- **Justification:** Nginx acts as the single entry point for all incoming traffic, forwarding requests to the Spring Boot application while handling HTTPS termination via the setup-https.sh script present in the repository. This separation improves security, enables efficient static asset serving, and makes it straightforward to add rate limiting or load balancing in future phases.
- **Key responsibilities:** Reverse proxy for the Spring Boot API, HTTPS/TLS termination, and static asset delivery.
- **Reference:** nginx.org/en/docs

Docker + Docker Compose (Containerisation & Orchestration)

- **Description:** Docker is a containerisation platform that packages applications and their dependencies into portable, isolated containers. Docker Compose is a tool for defining and running multi-container applications via a single YAML configuration file.
- **Justification:** The docker-compose.yml at the root of the repository orchestrates all services - the Spring Boot application, MySQL database, and Nginx reverse proxy - as a unified stack. This guarantees environment consistency between development and production, simplifies onboarding (single command startup), and makes the deployment reproducible with no manual environment configuration.
- **Key responsibilities:** Containerisation of all system services, local development environment setup, and production deployment orchestration.
- **Reference:** docs.docker.com - docs.docker.com/compose

GitHub Actions (CI/CD)

- **Description:** Integrated CI/CD automation platform built into GitHub that triggers configurable pipelines on repository events such as push or pull request.
- **Justification:** As the codebase is hosted on GitHub, GitHub Actions provides native, zero-configuration CI/CD integration. It enables automated build and test execution on every pull request and can trigger Docker image builds and deployment to the production host,

keeping the pipeline auditable and the deployment process repeatable without additional tooling costs.

- **Key responsibilities:** Automated build verification, test execution, Docker image build, and deployment promotion to the production environment.
- **Reference:** docs.github.com/actions